RAG-Based Document Search Server

Build a simple **Retrieval-Augmented Generation (RAG) application** where documents can be uploaded and indexed on a server, and users can search the available information through a chat interface.

The first version should focus only on the core functionality required to upload documents, index their content, retrieve relevant information, and answer user queries.

Input

- Documents uploaded to the server.
- Initially support common document formats such as:
    - PDF
    - DOCX
    - TXT
- Questions entered by users through the chat interface.

Output

- Answers generated from the uploaded documents.
- Relevant source document name should be displayed with the answer.
- Page number should also be displayed where it can be identified.

Core Requirements

1. Document Upload

Provide a simple interface where documents can be uploaded.

Store the original uploaded files in a folder on the server.

Use existing libraries wherever possible instead of developing custom document parsers.

2. Document Processing

When a document is uploaded:

1. Extract its textual content.
2. Divide the extracted text into smaller chunks.
3. Generate embeddings for the chunks.
4. Store the embeddings along with basic metadata such as:

- Document name
- Page number, where available
- Chunk text

3. Document Indexing

Use an existing vector database

ChromaDB

Store the generated embeddings and document metadata in a persistent local Chroma database.

The index should remain available after the application/server is restarted.

4. Embedding Generation

Use an existing embedding model.

5. Chat Interface

Provide a simple chat interface where users can enter questions in natural language.

Use **Streamlit** for the initial interface instead of developing a separate frontend application.

The interface should contain:

- Chat history
- Question input box
- Generated answer
- Source document information

6. Query Processing

When a user enters a question:

1. Generate an embedding for the query.
2. Search the vector database for the most relevant document chunks.
3. Retrieve a small number of the most relevant chunks.
4. Send the retrieved context along with the user question to the selected Large Language Model.
5. Generate the final answer.
6. Display the answer along with its source document information.

The basic flow will be:

User Question

↓

Embedding Generation

↓

Chroma Vector Search

↓

Relevant Document Chunks

↓

LLM

↓

Answer + Sources

7. Large Language Model

The RAG system should allow use of an existing Large Language Model

A locally hosted model using tools such as **Ollama**.

The final model should be selected based on server capability, privacy requirements, response quality, and available resources.

The LLM should be instructed to answer primarily from the retrieved document context.

If relevant information cannot be found in the uploaded documents, the system should clearly indicate that sufficient information was not found.

8. Source Information

Each generated answer should display basic source information.

Example:

Answer:
The admission test will be conducted on 15 November.

Sources:

- Admission Guidelines.pdf — Page 4
- Test Instructions.pdf — Page 2

9. Basic Document Management

Provide a simple page or section showing uploaded documents.

Initially, only the following actions are required:

- Upload document
- View uploaded documents
- Delete document

When a document is deleted, its indexed data should also be removed from the vector database.

Complex document versioning and approval workflows are not required in the initial version.

Basic Application Flow

Document Upload

Upload Document

↓

Extract Text

↓

Split into Chunks

↓

Generate Embeddings

↓

Store in ChromaDB

Document Search

User Question

↓

Search ChromaDB

↓

Retrieve Relevant Content

↓

Send Context + Question to LLM

↓

Display Answer + Sources